不详细揭秘 KMP 算法的原理

0. 记号

字符串的下标从 1 开始。$|s|$ 表示字符串 s 的长度。对于字符串 s，记其每一个字符分别为 $s_1, s_2, \cdots, s_{|s|}$。子串 $s_l, s_{l+1}, \cdots, s_{r-1}, s_r$ 简记为 $s[l, r]$。

对于字符串 a, b，$a + b$ 表示拼接操作，即将字符串 b 拼接到字符串 a 之后，构成新的字符串。
记构成的新字符串为 c，则上述拼接操作记为 $c \leftarrow a + b$。其中符号 $x \leftarrow y$ 表示将 y 的值赋给 x。

不论是字符还是字符串，皆不加引号。

1. Border 与周期的简单性质

对于字符串 s，若存在一个非本身的子串 t 使得 t 既是 s 的前缀又是 s 的后缀（但不能为 s 本身），称 t 为 s 的一个 Border。更加符号化地，对于 $|s| = n$，若存在 m 使得 $m \neq n$ 且 $s[1, m] = s[n - m + 1, n]$，称 $s[1, m]$ ($s[n - m + 1, n]$ 同理) 为 s 的一个长度为 m 的 Border。

对于一个字符串 s，记 $\mathrm{Border}(s)$ 为 s 的所有 Border 的集合， $\mathrm{MaxBorder}(s)$ 为 s 的 Border 中最长的一个（显然是唯一的）。

画图容易看出，若 s 有一个长为 m 的 Border，则 $|s| - m$ 是 s 的一个周期。注意这并不一定是整周期，如 abacab 有周期 4，而 abcabc 有周期 3，后者为整周期。

Periodicity Lemma（仅供参考，和下面 KMP 没关系）

若 p, q 均为字符串 s 的周期，且 $p + q - (p, q) \leq |s|$，则 (p, q) 也为 s 的周期。这个界是紧的，对于更小的情况一定可以构造出反例。如对 $p = 4, q = 6$ 满足 $p + q - (p, q) = |s| + 1$，存在反例 abacaba。

这里给一个简要证明思路（参考自 这里）。把 $\bmod q$ 相同的位置缩成一个点，连边 $i \to (i + p) \bmod q$。如果把边全连上（$i = 1, \cdots, q$）那么整张图形成 (p, q) 个长为 $\dfrac{q}{(p, q)}$ 的环。但因为字符串长度的限制，我们最多连到 $|s| - p \to |s|$。

把 q 个点连成 (p, q) 个连通块需要 $q - (p, q)$ 条边，最后一条边为 $q - (p, q) \to p + q - (p, q)$。于是 $p + q - (p, q) \leq |s|$ 时即可成功连上所有的边，形成 (p, q) 个连通块，即 (p, q) 的周期。很明显少连边我们可以将不连通的点赋予不同字符，于是界是紧的这一点容易理解。

$p + q \leq |s|$ 时称 Weak Periodicity Lemma，此时原图连通块全都是环。

2. 前缀函数、KMP

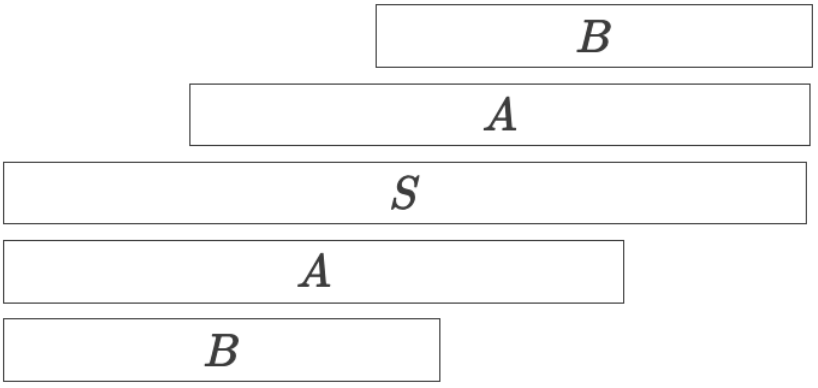
对于字符串匹配问题，设模式串为 s，文本串为 t，我们构造新字符串 $s + \# + t$，其中 $\#$ 是 s, t 中均不存在的字符。此时对于匹配位置 p，即前缀 $[1, p]$ 的最长 Border 长度为 $|s|$，且这个条件是充要的。注意实际上我们不只能算出 s 的匹配位置，所有 s 的前缀的匹配位置都可用完全一致的方法得到。

于是对于一个字符串 s，定义前缀函数 $\pi(s, i) = |\mathrm{MaxBorder}(s[1, i])|$，在不引起歧义的情况下简记为 $\pi(i)$。特别地，定义 $\pi(s, 1) = 0$。对于一整个字符串的前缀函数值我们记 $\pi(s) = \{\pi(s, 1), \pi(s, 2), \cdots, \pi(s, |s|)\}$，如 $\pi(\mathtt{abacaba}) = \{0, 0, 1, 0, 1, 2, 3\}$。

于是问题变成计算前缀函数，考虑递推。对于位置 $i + 1$，容易看出这里的 Border 由 i 位置的一个 Border 继承而来：i 位置的 Border 为 $s[1, j]$， $s_{j+1} = s_{i+1}$。

于是我们就有了思路。从长到短遍历 i 位置的 Border，从第一个满足 $s_{j+1} = s_{i+1}$ 的位置得到 $\pi(i + 1) = j + 1$。

问题来了我们怎么遍历 i 位置的所有 Border。注意到，次长 Border 是最长 Border 的最长 Border（图），于是我们直接用已经算出来的 π 遍历即可。



看代码或许直观一些。

```
void prefix()
{
    int j=0;
    for(int i=2;i<=n;i++)
    {
        while(j>0 && s[i]!=s[j+1]) j=pi[j];
```

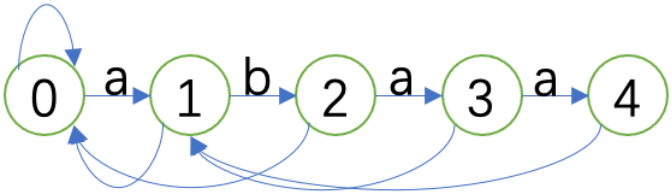
```
        if(s[i]==s[j+1]) j++;
        pi[i]=j;
    }
}
```

分析时空复杂度。对于求前缀函数，空间复杂度显然为 $O(|s|)$。对于时间复杂度，即 j 的跳跃次数，注意 j 向右跳（ j++ ）的距离不超过 $|s|$ ，并且由于 $j>0$ 恒成立 j 向左跳的距离不超过向右跳的距离，于是 j 跳的总距离不超过 $|s|$，时间复杂度 $O(|s|)$，每字符均摊 $O(1)$。

拼接 $s+\#+t$ 做字符串匹配时，注意只有 s 的前缀函数是有用的，我们逐位算 t 的时候算完一位就可以扔掉一位，于是空间复杂度仍为 $O(|s|)$，时间复杂度为 $O(|s|+|t|)$，每字符均摊 $O(1)$。

3. KMP 的第二种写法，KMP 自动机

上面拼接字符串的写法和市面上常见的 KMP 写法略有不同。市面上常见的 KMP 写法简单来说就是下面的图。模式串 abaa 算出前缀函数，然后按照前缀函数的值连边。



具体解释一下，$\pi(\texttt{abaa})=\{0,0,1,1\}$，图中编号为 i 的点表示已经成功匹配了前 i 个字符。i 向编号为 $\pi(i)$ 的点连有向边，同时向 $i+1$ 连 s_{i+1}。

给定文本串（如 aabaaabaa） 时，逐位转移。

- 一开始位于空位置 0，表示未匹配任何字符。
- 增加 a，按照横着的边转移到 1，表示匹配了 a。
- 再增加 a，此时失配，沿下方边回到 0，再沿 a 到 1。
- 依次增加 baa，转移到 4，表示匹配了 abaa。
- 增加 a，失配先到 1 再到 0，再沿 a 到 1。
- 依次增加 baa，转移到 4，表示匹配了 abaa。

本质上这种写法和拼字符串是一模一样的，只是把跳 j 画到了图上。

这个做法（以及之前拼字符串的做法）有一个问题，就是复杂度的均摊特性。如果我们只是一位一位地匹配当然是没有问题的，但这难以支持撤销操作。

具体来说，考虑模式串 $s=\texttt{aaaaaa}$，文本串 $t=\texttt{aaaaaa}$，此时转移到最后一个位置。接下来反复增加字符 b 并撤回，j 的位置会不断地从末尾跳 $|s|$ 次到开头，再回到末尾，单次时间复杂度变为 $O(|s|)$，总时间复杂度可达 $O(k|s|)$，其中 k 为操作次数，远远差于先前的单次均摊 $O(1)$。

解决方法非常简单，我们建立上面的图之后提前算出所有可能字符的转移，即从当前点开始增加一个字符会到达的下一个位置。这样预处理的时空复杂度均为 $O(|s|\Sigma)$，其中 Σ 为字符集大小，后续匹配 / 撤回操作单次时间复杂度 $O(1)$。 改造后的这个图被称为 KMP 自动机。

> 什么是自动机？ >
>
> 这里的自动机指有限状态自动机（DFA）。简单来说就是一张有向图，结点称为状态（Q），而从一个状态到另一个状态的有向边称为转移。转移方式由转移函数（δ）进行规定。
> DFA 有一个起始状态（$start$）和若干个接受状态（F）。当我们把规定字符集（Σ）中字符组成的字符串输入该 DFA 时，DFA 会从起始状态开始，依照转移函数，逐个字符进行转移。
> 若最终停在了接受状态，称该 DFA 接受该字符串；否则称该 DFA 不接受该字符串。
> 有一个空状态 null，若从 u 状态由字符 c 不能转移到任何一个非空状态，则转移到空状态。null 不是接受状态，且不能转移到接受状态，只能转移到 null。
>
> 对于上面的 KMP 自动机，如果仅规定最后一个点为接受状态，则其接受且仅接受所有以模板串 s 为后缀的字符串；如果规定除初始点外所有点均为接受状态，则其接受且仅接受所有以模板串 s 的某个前缀为后缀的字符串。

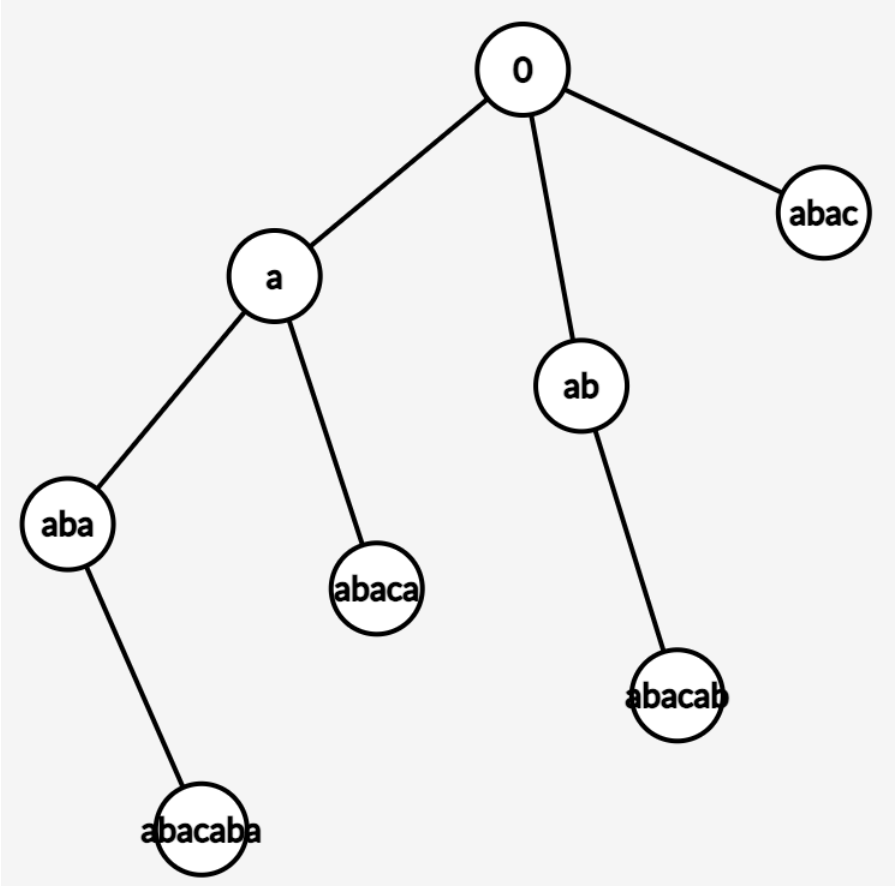
4. 一点简单应用

字符串周期统计

根据第一节的讨论等价于求字符串的所有 Border，跳前缀函数即可。

Border 树

所有的前缀通过前缀函数构成了一个树形结构（Border 树 / 失配树）：



- 前缀的最长公共 Border 等价于最近公共祖先。
- 前缀的出现次数就是上面 Border 树的子树大小（前缀在后面出现 $\iff$ 成为一个前缀的 Border）。求这个无需显式建出树。